《面向对象程序设计》

期末串讲

刘润林　林艺涵

期末考试

- 闭卷考试，上机考试
- 2025年1月7日下午13:20-15:20，提前30分钟进入考场
- 单选题(30)、多选题(5)、编程题&简答题(4)
- 主要考试内容
  - 面向对象核心思想（e.g., 设计原则）
  - 封装（e.g., 目的、访问范围、重载、静态属性……）
  - 继承（继承的优缺点、重写/覆盖、转型、super……）
  - 多态（e.g., 多态的必要条件、表现形式……）
  - 设计原则&设计模式
  - 其他内容：集合、异常、多线程、网络、I/O（各有1-2道选择题）

Java基础

:)

导论

编程语言分类：脚本语言、面向机器语言、面向过程语言、**面向对象语言**

程序运行方式：编译执行、解释执行、**编译和解释相结合（如 Java）**

面向对象思想

万事万物皆对象，

程序由类组成，

对象是类的实例、是细粒度的，

对象之间通过消息相互作用。

Object-Oriented

“物件导向”

类和对象之间是**抽象**和**具体**的关系。

类是创建对象的模板，对象是类的具体实例。

关键字和保留字

51 个关键字

基本数据类型、类型的大致范围、越界问题、强转问题

boolean , char , byte , float , int , long , short , double , void , enum

程序控制流

do , while , for , if , else , break , continue , return , switch , case , default

类及接口

class , interface , private , protected , public , implements , extends

修饰词

static , abstract , final , transient , synchronized , volatile , strictfp , native

try , catch , finally , throw , throws , assert , instanceof **异常、断言、类型判断**

文件组织

import , package

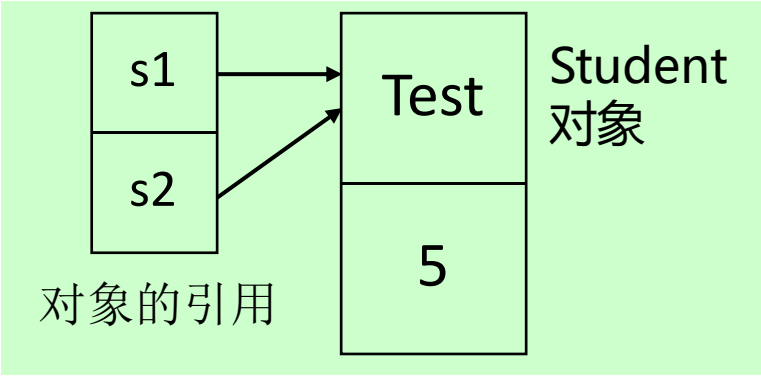
new, super, this

3 个直接量 true , false , null

2 个保留字 goto , const

引用数据类型

Student s1 = new Student(“Test”, 5);
Student s2 = s1;



s1和s2指向同一个对象

```
public static void main(String[] args) {
    Student s1 = new Student("Test", 5);
    change(s1);
    System.out.println(s1.age);
}
public static void change(Student student){
    student.age = 10;
}
```

输出 **10**！

构造（Constructor）与垃圾回收

特点：与类同名，无返回值（也不是 void）、多数情况要重载

new 的作用：**分配空间；调用构造；返回引用**

垃圾自动回收机制（Garbage Collection）：Java 虚拟机后台线程负责

System.gc() 和 Runtime.gc()

判断存储单元是否为垃圾的依据：引用计数为 0

匿名对象：无管理者（无栈内存引用指向它）

使用场景：只需要进行一次方法调用 / 作为参数传递给函数

封装

Encapsulation :)

封装（Encapsulation）

修饰符	同一个类	同一个包	子类	整体
private	**可见**	不可见	不可见	不可见
default / 没有	**可见**	**可见**	不可见	不可见
protected	**可见**	**可见**	**可见**	不可见
public	**可见**	**可见**	**可见**	**可见**

protected

（1）包内可见。

（2）对子类可见（子类和父类在同一个包：通过自己访问、通过父类访问。在不同包：仅可通过自己访问。）

（3）protected **的** static **成员对所有子类可见。**

2. 以下关于构造方法的说明不正确的是：_____

A 构造方法名必须和类名相同

B 构造方法不返回任何值，也没有返回类型

C 每一个类只能有一个构造方法

D 构造方法在创建对象时调用，其他地方不能显式地直接调用

正确答案: C

8. 下列关于权限的说法中错误的是：_____

A Java中一共有四种访问权限控制，其权限的大小顺序：public > protected > default > private

B 普通方法可以使用四种访问权限，但是抽象方法不能使用private来修饰

C 构造器可以使用四种访问权限

D protected所修饰的成员，对于同包子类可访问，但是外包子类不可访问

正确答案: D

static

静态成员变量：类加载时就被装载和分配，不依赖于对象而存在 => 通过 类名.变量名 访问

静态成员方法：可以通过 **类名.函数名 调用，只能访问类的静态成员**

静态代码块：仅能定义在类中，一般用于初始化类的静态变量

语句执行顺序

第一次 new 时：**（1）初始化有显式初始化的静态成员变量；（2）顺序执行静态代码块；**（3）初始化有显式初始化的非静态成员变量；（4）顺序执行非静态代码块；（5）调用构造。

之后再 new 时：（1）初始化有显式初始化的非静态成员变量；（2）顺序执行非静态代码块；（3）调用构造。

完善类的设计

equals 和 ==

- 比较**对象的equals和==是等价的**，判断是不是引用的同一个对象。
- **String的equals**只看字符串内容是否相等，而==还得看**是不是同一个对象**。（覆盖了Object类的equals()方法，Date类同理）

```
String str1=new String("Hello");
String str2=new String("Hello");

System.out.println(str1==str2); //打印false
System.out.println(str1.equals(str2)); //打印true
```

内部类

内部类的一个特点是能够访问外部类的各种访问级别的成员。

- 实例内部类：在创建实例内部类的实例时，外部类的实例必须已经存在。实例内部类可以直接访问外部类的所有东西。
- 静态内部类：创建实例时不必创建外部类的实例。
- 局部内部类：只能在当前方法中使用，可以访问外部类的所有成员；还可以访问方法中的final参数或最终变量。

方法重载（Overload）

方法重载：Java允许在一个类中定义多个同名的方法，但这些方法的**参数列表必须不同（参数个数、参数类型及参数顺序至少有一个不同）**

```java
public void print(String name, int age) {
    System.out.println("Name: " + name + ", Age: " + age);
}
public void print(int age) {
    System.out.println("Age: " + age);
}
```

返回值类型与**访问权限修饰符可以相同也可以不同**，上述两项不能当做判断是否 重载的条件。

```java
private void show(String msg) {
    System.out.println("Private method: " + msg);
}
public void show(String msg) {
    System.out.println("Public method: " + msg);
}
```

3. 当判断一个类中的两个方法是不是重载方法时，不需要考虑：_____

A 返回值类型

B 参数个数

C 参数顺序

D 参数类型

正确答案: A

继承

Inheritance :)

继承（Inheritance）

继承虽好，但也破坏了封装性。子类(派生类)拥有父类(基类/超类)的**所有属性和方法**(不过私有的属性和方法不能直接访问)

构造方法不能继承

子类构造方法必须最先调用父类构造。

new 出子类对象时，构造方法按照继承链从上往下依次调用。

变量隐藏：子类隐藏父类的同名变量。

继承：super vs this

共性

（1）出现在实例方法和构造方法中

（2）不能出现在静态方法中

（3）大部分情况下是可以省略的

（4）this(), super() 只能出现在构造方法的第一行（若没有 自动补上无参构造方法）

super

（1）当在子类对象中，子类想访问父类的东西，可以使用“super.”的方式访问

（2）当子类构造方法的第一行执行super()无参数方法，那么父类中一定要有无参数构造方法（当一个类的构造方法是显式的有参数的，会替代原本的无参构造方法，这个时候如果子类执行super()无参方法就会报错，一个好的习惯是要补上无参的构造方法）

继承：方法重写 / 覆盖（Override）

条件

（1）子类中方法的返回值必须与父类的相同、或是其子类。

（2）**方法名、形参列表**完全相同。

（3）**访问权限**不能更低，只能相同或更高。

（4）抛出异常的范围不能更大。

注意事项

（1）**private 方法**不能继承，所以不能覆盖，只有隐藏。

（2）**构造方法**不能继承，所以不能覆盖。

（3）**静态方法**不存在覆盖，只有隐藏。

（4）**final 修饰**的是最终方法，不能覆盖。

4. 关于方法重写的特点，下列错误的是：_____

A 重写的方法的返回值类型应该与父类中被重写方法的返回值类型相同或是他的子类类型

B 重写的方法不能抛出比父类更加宽泛的异常

C 重写的方法必须方法名相同，参数列表也相同

D 重写的方法的访问权限应该比父类中被重写方法的访问权限高

正确答案: D

5. 关于方法重写的注意事项中，下列正确的是：_____

A 构造方法可以被重写

B 方法重写是父类与子类之间多态性的一种表现

C final修饰的方法不能被重写，但static修饰的方法能被重写

D private修饰的成员方法能被重写

正确答案: B

多态

Polymorphism :)

多态（Polymorphism）

静多态：发生在编译时，一般指方法重载。

动多态：发生在运行时，一般指动态调用方法。

动多态的三个条件：继承、覆盖、向上转型（父类引用管理子类对象）

多态中成员变量：编译运行看左边

多态中成员方法：编译看左边，运行看右边。

向上转型 upcasting

访问子类继承或隐藏的成员变量，也可以调用子类继承的方法或子类重写的实例方法。

上转型对象**不能**操作子类新增的成员变量；**不能**调用子类新增的方法。

父类类型 变量名 = new 子类类型();

```
Person p = new Student();
```

安全的

向下转型 downcasting

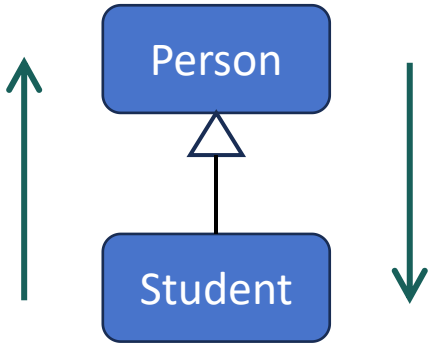
一个已经向上转型的子类对象可以使用强制类型转换的格式，将父类引用转为子类引用。

子类类型 变量名 = (子类类型) 父类类型的变量

```
Person p = new Student();
Student stu = (Student) p
```

```
Person p = new Peron();
Student stu = (Student) p
```

如果是直接创建父类对象，会产生运行时异常



不一定安全

使用**instanceof**判断

7. 下列说法错误的是：_____

A 多态的前提是必须有子父类关系或者类实现接口关系

B 多态在子父类关系中其实是一种向下转型

C 使用多态后的父类引用变量在调用方法时，会调用子类重写后的方法

D 编译时多态是静态的多态，运行时多态是动态的多态

正确答案: B

9. 关于向上映射和向下映射的说法错误的是_____

A 对象的向上映射总是安全的，可靠的

B 对象的向下映射如果不可以转的话，程序会产生ClassCastException，这种异常是编译时可监测到的异常，必须处理。

C 上转型对象可以访问子类继承或隐藏的成员变量

D 上转型对象不能操作子类新增的成员变量，不能调用子类新增的方法

正确答案: B

抽象类与接口

关键词 **abstract** 的语义：尚未实现。
任何包含抽象方法的类必须被声明为抽象类，必须用 abstract 修饰。抽象类不能实例化。

抽象方法

（1）无函数体（区别函数体为空）

（2）必须在抽象类中

（3）必须在子类中实现，除非子类也是抽象的

抽象方法**不能被 private final static 修饰**。

interface：方法特征集合，类的行为规范。

接口中所有方法&变量，默认、也必须是 public abstract 的。
接口没有构造方法。

接口回调：用接口类型的引用管理实现了该接口的类。
接口可以继承：同一个函数只能实现一次；不同接口的同名变量相互隐藏；接口变量和类中成员同名时，存在作用域问题。

其他知识

others :)

多线程

Thread	Runnable
简单易用	编码较复杂
java不支持多继承，因此有局限性	可以避免单继承的限制

Sleep	yield
使线程转入阻塞状态	使线程转入runnable状态
不会考虑线程的优先级	相同优先级或更高的线程运行机会

wait, notify, notifyAll必须在已经持有锁的情况下执行,所以它们只能出现在**synchronized**作用的范围内，也就是出现在**synchronized**修饰的方法中。

wait的作用:**释放已持有的锁**,进入等待队列.

notify的作用:**随机唤醒**wait队列中的一个线程并把它移入锁申请队列

notifyAll的作用:唤醒wait队列中的所有的线程并把它们移入锁申请队列

单例模式

饿汉式

是否 Lazy 初始化：否

是否多线程安全：是

常用，但容易产生垃圾对象。

优点：没有加锁，执行效率会提高。

缺点：类加载时就初始化，浪费内存。

特点：避免了多线程的同步问题

懒汉式

是否 Lazy 初始化：是

是否多线程安全：否

不支持多线程。因为没有加锁 synchronized

```
//静态的。保留自身的引用。
private static Singleton test = null;
//必须是私有的构造函数
private Singleton(){}
//公共的静态的方法。
public static Singleton getInstance()
```

6. 关于单例模式，以下说法正确的是：_____

A 它拥有公有的构造方法

B 它拥有公有实例方法getInstance()，以获得唯一的一个实例

C 懒汉式单例，即单例实例在第一次被使用时构建；而饿汉式单例，即JVM加载这个类时就创建单例实例

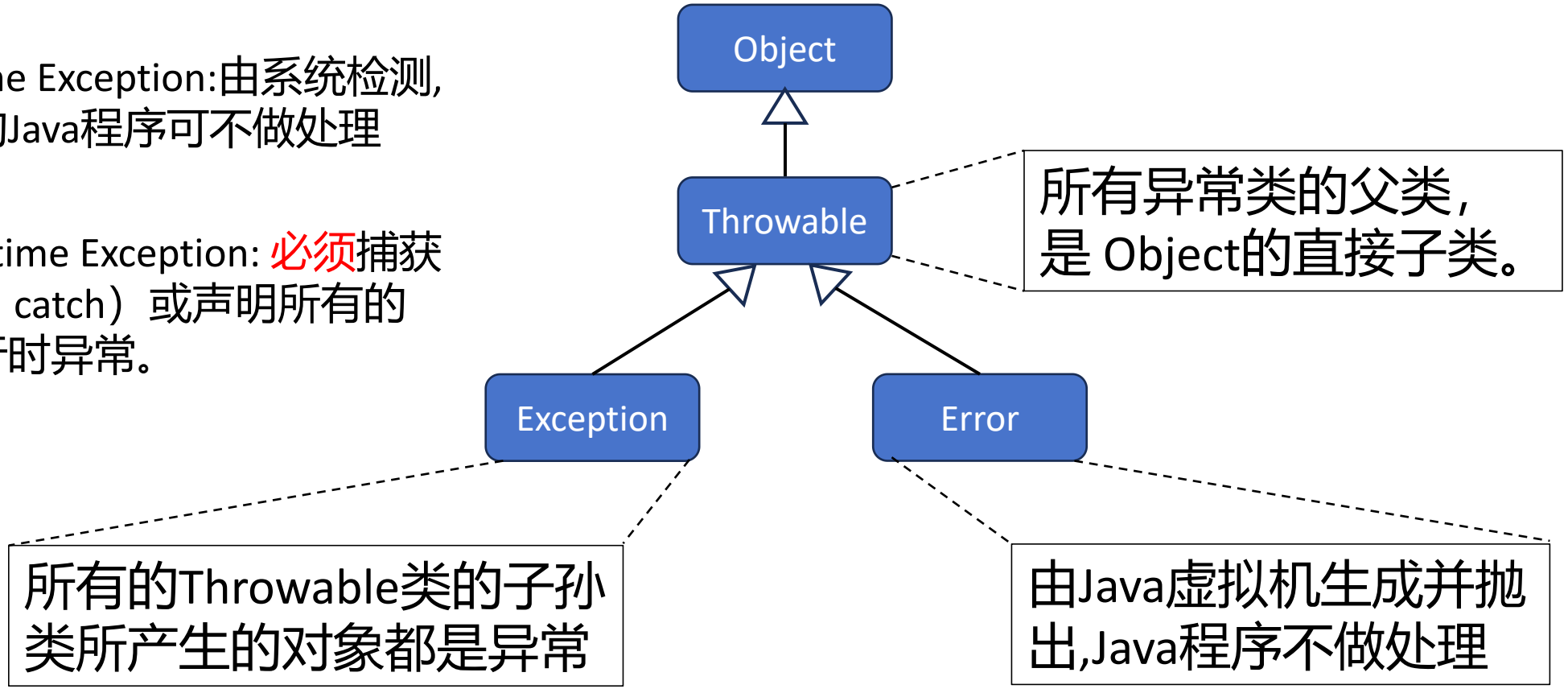
D 其它选项均不正确

正确答案: C

异常处理

Runtime Exception:由系统检测,用户的Java程序可不做处理

非Runtime Exception: 必须捕获（try，catch）或声明所有的非运行时异常。



Copyright (C) Tony's Studio 2023

网络编程

- C/S， B/S
- URI， URL， TCP， UDP
- Socket
- …

了解基本概念即可，不会考太难。

Copyright (C) Tony's Studio 2023

Design Patterns

Where the fun begins. :)

设计原则

- **单一职责**原则

一个类只负责一个功能领域中的相应职责。

eg. 有一个图形绘制类，它既负责绘制圆形又负责绘制矩形和三角形等多种图形，这样当需要修改圆形的绘制算法时，可能会影响到其他图形的绘制。按照单一职责原则，可以将其拆分成圆形绘制类、矩形绘制类和三角形绘制类等，每个类只负责一种图形的绘制，这样修改其中一个类时不会影响到其他类

- **开闭原则**

一个软件实体应当对扩展开放，对修改关闭。

```java
class GraphicDrawer {
    public void drawShape(Shape shape) {
        if (shape instanceof Circle) {
            // 绘制圆形的代码
        } else if (shape instanceof Rectangle) {
            // 绘制矩形的代码
        }
    }
}
```

```java
class Circle implements Shape {
    // 圆形的属性和方法
}

class Rectangle implements Shape {
    // 矩形的属性和方法
}
```

设计原则

- **合成复用**原则

尽量使用组合或者聚合关系实现代码复用，少使用继承。

继承的耦合度很高

- **依赖倒转**原则

抽象不应该依赖于细节，细节应当依赖于抽象。换言之，要针对接口编程，而不是针对实现编程。

```
class Computer {
    private CPU cpu;
    private Memory memory;

    public Computer() {
        this.cpu = new IntelCPU();
        this.memory = new KingstonMemory();
    }
}

class IntelCPU {
    // CPU
}

class KingstonMemory {
    // 内存
}
```

```
interface CPU {
    void process();
}

interface Memory {
    void storeData();
}

class Computer {
    private CPU cpu;
    private Memory memory;

    public Computer(CPU cpu, Memory memory) {
        this.cpu = cpu;
        this.memory = memory;
    }
}
```

设计原则

- **接口隔离**原则

使用多个专门的接口，而不使用单一的总接口，即客户端不应该依赖那些它不需要的接口。

- **迪米特法则**

一个软件实体应当尽可能少地与其他实体发生相互作用。

- **里氏代换**原则

所有引用基类（父类）的地方必须能透明地使用其子类的对象

设计模式

- 创建型

  单例模式，工厂模式（简单工厂、抽象工厂），原型模式

- 结构型

  适配器模式，装饰模式，门面模式

- 行为型

  策略模式，访问者模式，责任链模式，观察者模式

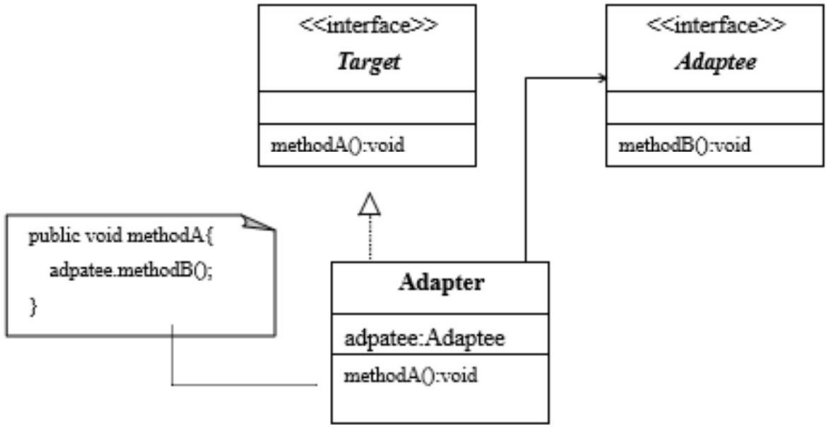
创建型

- 原型模式

用一个已经创建的实例作为原型，通过复制该原型对象来创建一个和原型相同或相似的新对象

实现了一个原型接口，该接口用于创建当前对象的克隆

结构型



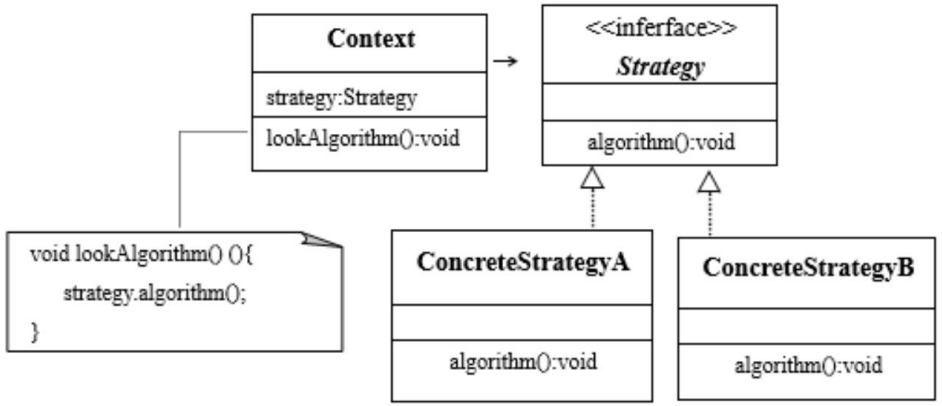
- 适配器模式

将一个类的接口转换成客户希望的另外一个接口实现了一个原型接口，该接口用于创建当前对象的克隆

适配器模式中包括三种角色：

- **目标（Target）：**目标是一个接口，该接口是客户想使用的接口。
- **被适配者（Adaptee）：**被适配者是一个已经存在的接口或抽象类，这个接口或抽象类需要适配。
- **适配器（Adapter）：**适配器是一个类，该类实现了目标接口并包含有被适配者的引用，即适配器的职责是对被适配者接口（抽象类）与目标接口进行适配。

行为型



- 策略模式

核心就是将类中经常需要变化的部分分割出来，并将每种可能的变化对应地交给抽象类的一个子类或实现接口的一个类去负责，从而让类的设计者不去关心具体实现

结构中包含以下三种角色：

- **策略（Strategy）**：策略是一个接口，该接口定义若干个算法标识，即定义了若干个抽象方法。
- **具体策略（ConcreteStrategy）**：具体策略是实现策略接口的类。具体策略实现策略接口所定义的抽象方法，即给出算法标识的具体算法。
- **上下文（Context）**：上下文是依赖于策略接口的类（是面向策略设计的类），即上下文包含有用策略声明的变量。上下文中提供一个方法，该方法委托策略变量调用具体策略所实现的策略接口中的方法。

UML图

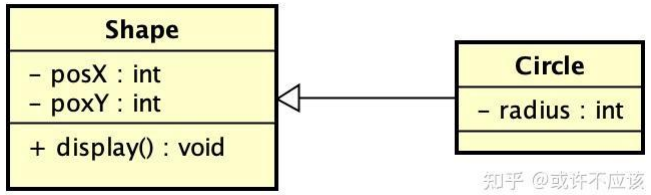
要会看 :)

UML图

- **泛化**

是一种**继承关系**，表示子类继承父类的所有特征和行为

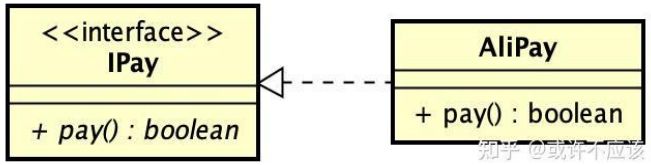
带**空心三角箭头的实线**，箭头指向父类



- **实现**

是一种类与接口的关系，表示类是接口所有特征和行为的实现

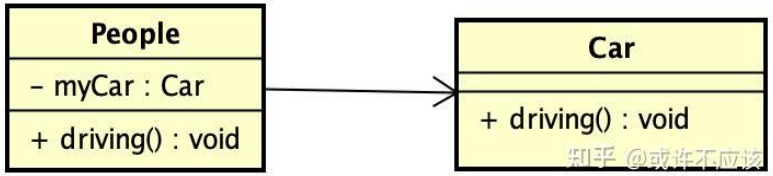
带**空心三角箭头的虚线**，箭头指向接口



UML图

- **关联**

是一种拥有关系，它使得一个类知道另一个类的属性和方法

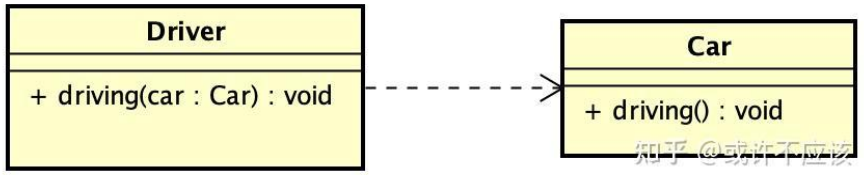


带**普通箭头的实线**，指向被拥有者。双向的关联可以有两个箭头，或者没有箭头。单向的关联有一个箭头

- **依赖**

是一种使用关系，即一个类的实现需要另一个类的协助

带**普通箭头的虚线**，普通箭头指向被使用者



Java集合框架

要会用 :)

Java集合框架

所有集合类都位于 java.util 包下。Java的集合类主要由两个接口派生而出：Collection 和 Map

Set、 List 和 Map 可以看做集合的三大类：

Set 接口继承 Collection，无序集合，集合中的元素不可以重复

List 接口继承 Collection，允许重复

Map 集合中保存 key-value 对形式的元素

1. 关于Java类LinkedList的特点，下面描述正确的是（）_____

A 查询快

B 增删快

C 元素不重复

D 元素自然排序

正确答案: B

10. 下面Java中关于List、Set的说法正确的是（）_____

A List 接口存储一组唯一，有序的对象

B Set 接口存储一组唯一，有序的对象

C Set可以允许插入多个null值

D ArrayList和Vector都继承自List

正确答案: D

ArrayList

ArrayList<String> sites = new ArrayList<String>();

添加元素：sites.add(“str”)

访问元素：sites.get(index)

修改元素：sites.set(2, "Wiki");

删除元素：sites.remove(index);

使用Collections排序：Collections.sort(sites); // 默认数字/字母排序

HashSet

HashSet<String> sites = new HashSet<String>();

添加元素：sites.add(“str”)

判断元素是否存在：sites.contains("success")

删除元素：sites.remove("success");

使用Collections排序：先转成List再用ArrayList的方法排序

List<String> sortedList = new ArrayList<>(sites);

*也可以是key-value的，比如HashSet<String, String>，第一个做key，第二个做value

Any questions?

预祝大家期末取得**好成绩！**

May the Force be with you!